

MEF - Modularización

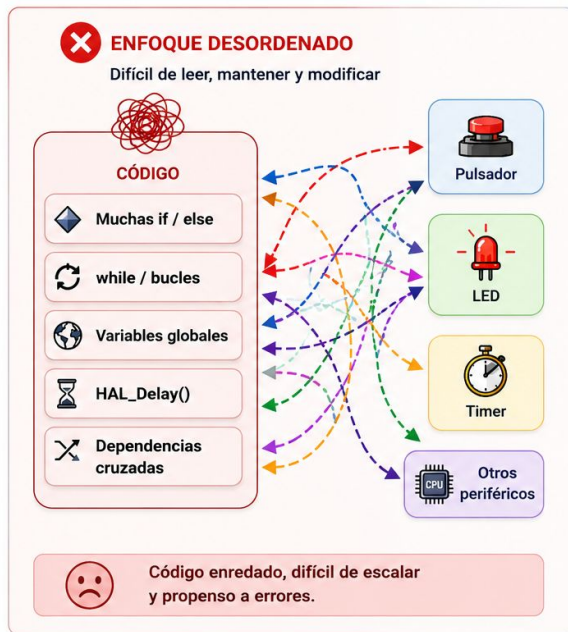
Clase: 26/05/2026

Objetivo de la clase

- Explicar qué es una Máquina de Estados Finitas y por qué resulta útil en sistemas embebidos.
- Identificar estados, eventos, transiciones y acciones en un problema embebido simple.
- Representar una FSM mediante un diagrama de estados.
- Implementar una FSM básica en lenguaje C.
- Diferenciar código bloqueante de código organizado por eventos.
- Comprender la importancia de separar el programa en módulos.
- Proponer una organización mínima de archivos .c y .h para un proyecto embebido.

¿Por qué necesitamos organizar el programa?

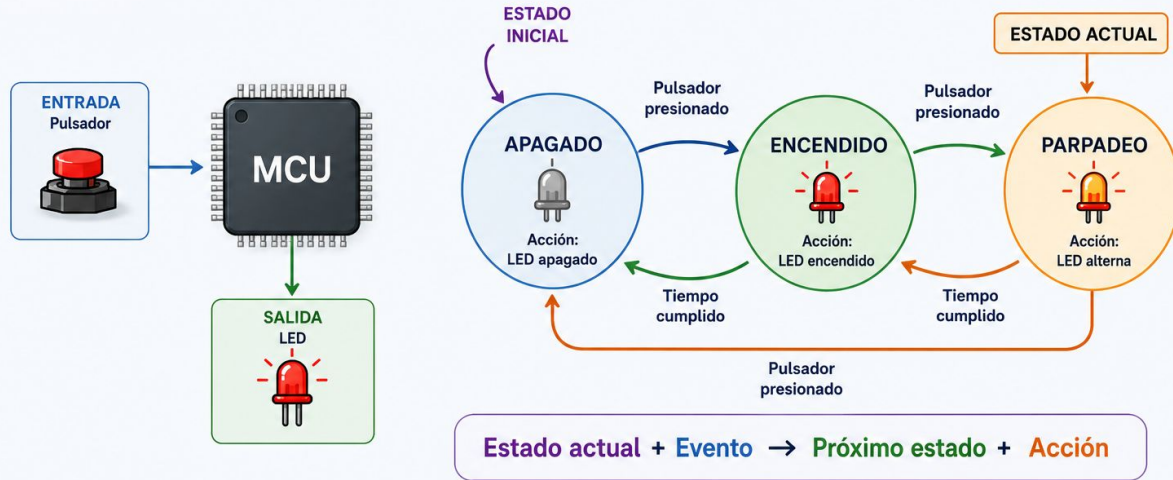
Control de LED con Pulsador



Concepto de Máquina de Estados Finitas

MÁQUINA DE ESTADOS FINITAS (FSM)

Modelo simple para controlar el comportamiento de un sistema embebido.



LEYENDA



ESTADO

Situación en la que se encuentra el sistema.

EVENTO

Ocurre de forma externa o interna y provoca una transición.



TRANSICIÓN

Cambio de un estado a otro, disparado por un evento.



ACCIÓN

Lo que el sistema hace mientras permanece en un estado.



ESTADO INICIAL

Estado desde donde comienza el sistema al encenderse.



Las FSM simplifican programas, mejoran la legibilidad y facilitan el mantenimiento.

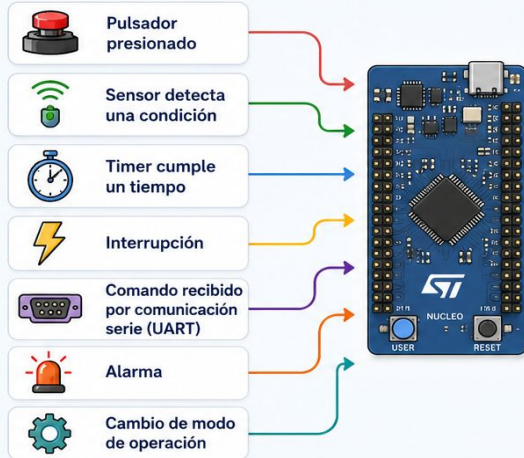
Ejemplo: Control de un LED con un pulsador

MEF en Sistemas Embebidos

¿POR QUÉ USAR MÁQUINAS DE ESTADOS FINITAS (FSM)?

En sistemas embebidos, el comportamiento depende de eventos que ocurren en cualquier momento.

EVENTOS QUE PUEDEN OCURRIR



MÁQUINA DE ESTADOS FINITOS (FSM)



Los sistemas embebidos reaccionan a eventos.

Las FSM ordenan esa reacción mediante estados y transiciones.

BENEFICIOS DE USAR FSM



Código organizado y modular



Fácil de entender y mantener



Escalable: fácil agregar nuevos estados

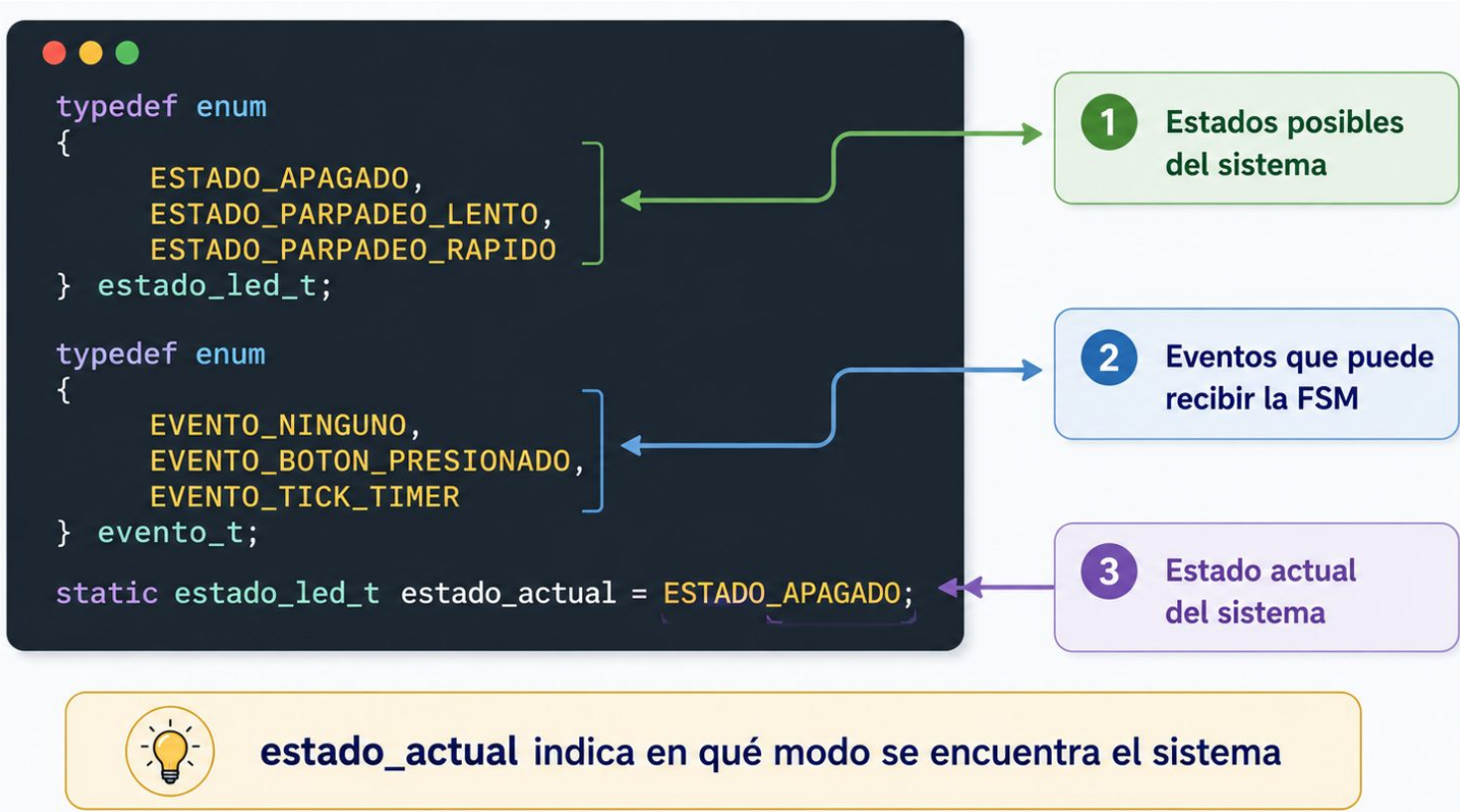


Comportamiento predecible y robusto



Ideal para sistemas reactivos y basados en eventos

Representación de estados y eventos en C



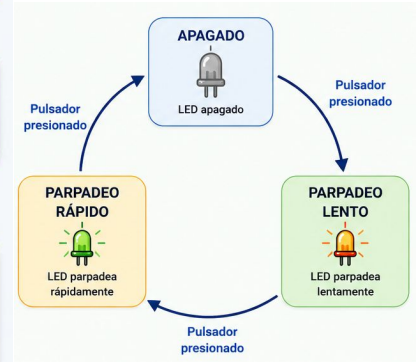
Función de transición: procesar eventos

```
void led_fsm_procesar_evento(evento_t evento)
{
    switch (estado_actual)
    {
        case ESTADO_APAGADO:
            if (evento == EVENTO_BOTON_PRESIONADO)
            {
                estado_actual = ESTADO_PARPADEO_LENTO;
            }
            break;
        case ESTADO_PARPADEO_LENTO:
            if (evento == EVENTO_BOTON_PRESIONADO)
            {
                estado_actual = ESTADO_PARPADEO_RAPIDO;
            }
            break;
        case ESTADO_PARPADEO_RAPIDO:
            if (evento == EVENTO_BOTON_PRESIONADO)
            {
                estado_actual = ESTADO_APAGADO;
            }
            break;
        default:
            estado_actual = ESTADO_APAGADO;
            break;
    }
}
```

1 switch: evalúa el estado actual

2 if: evalúa el evento recibido

3 La transición modifica estado_actual



Estado actual + evento recibido → próximo estado

Acciones asociadas al estado

```
void led_fsm_actualizar_salida(void)
{
    switch (estado_actual)
    {
        case ESTADO_APAGADO:
            led_apagar();
            break;
        case ESTADO_PARPADEO_LENTO:
            led_parpadear(1000);
            break;
        case ESTADO_PARPADEO_RAPIDO:
            led_parpadear(200);
            break;
        default:
            led_apagar();
            break;
    }
}
```

1

Si el estado es
APAGADO → apagar LED

2

Si el estado es
PARPADEO LENTO →
parpadear cada 1000 ms

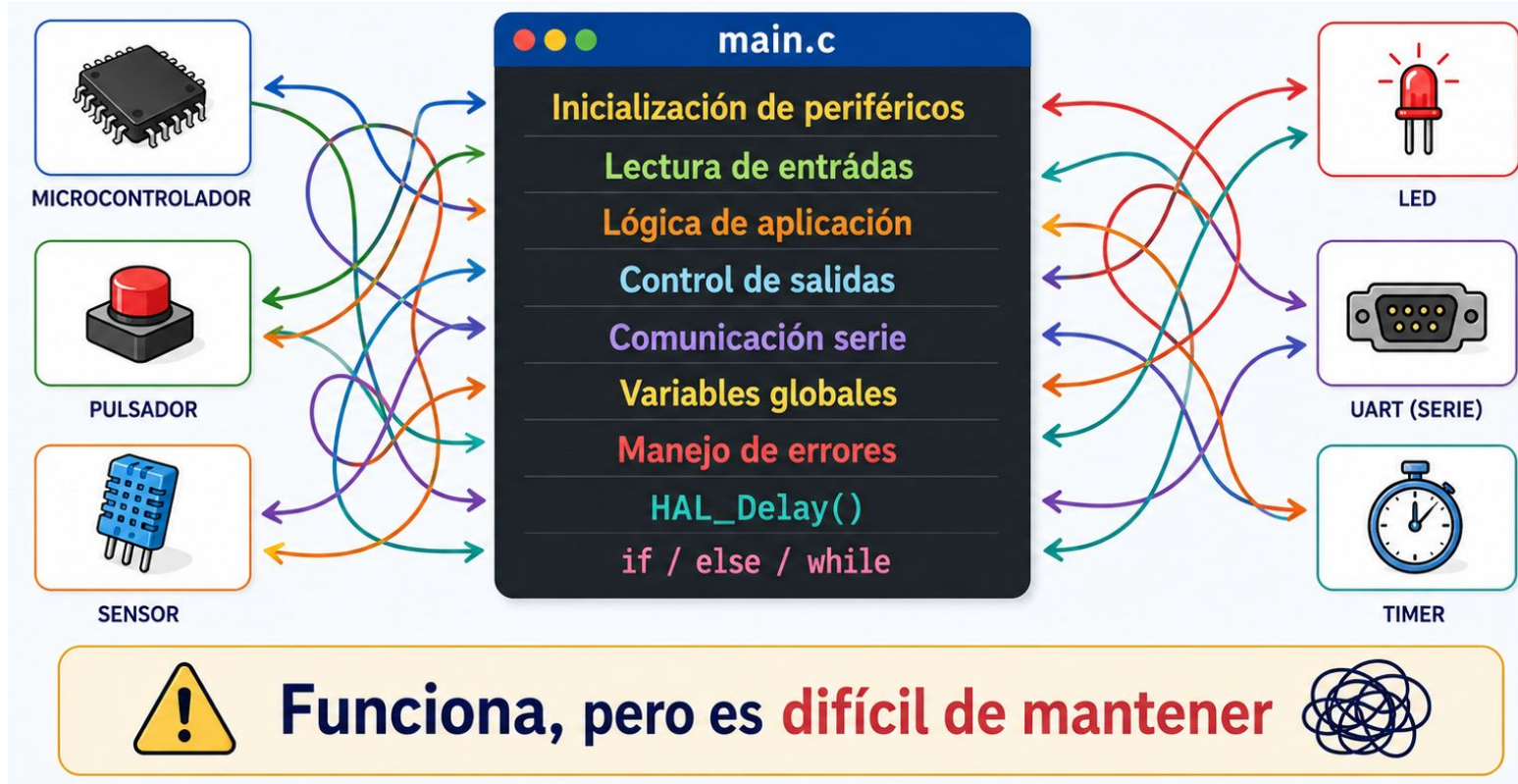
3

Si el estado es
PARPADEO RÁPIDO →
parpadear cada 200 ms



Estado actual → acción sobre la salida

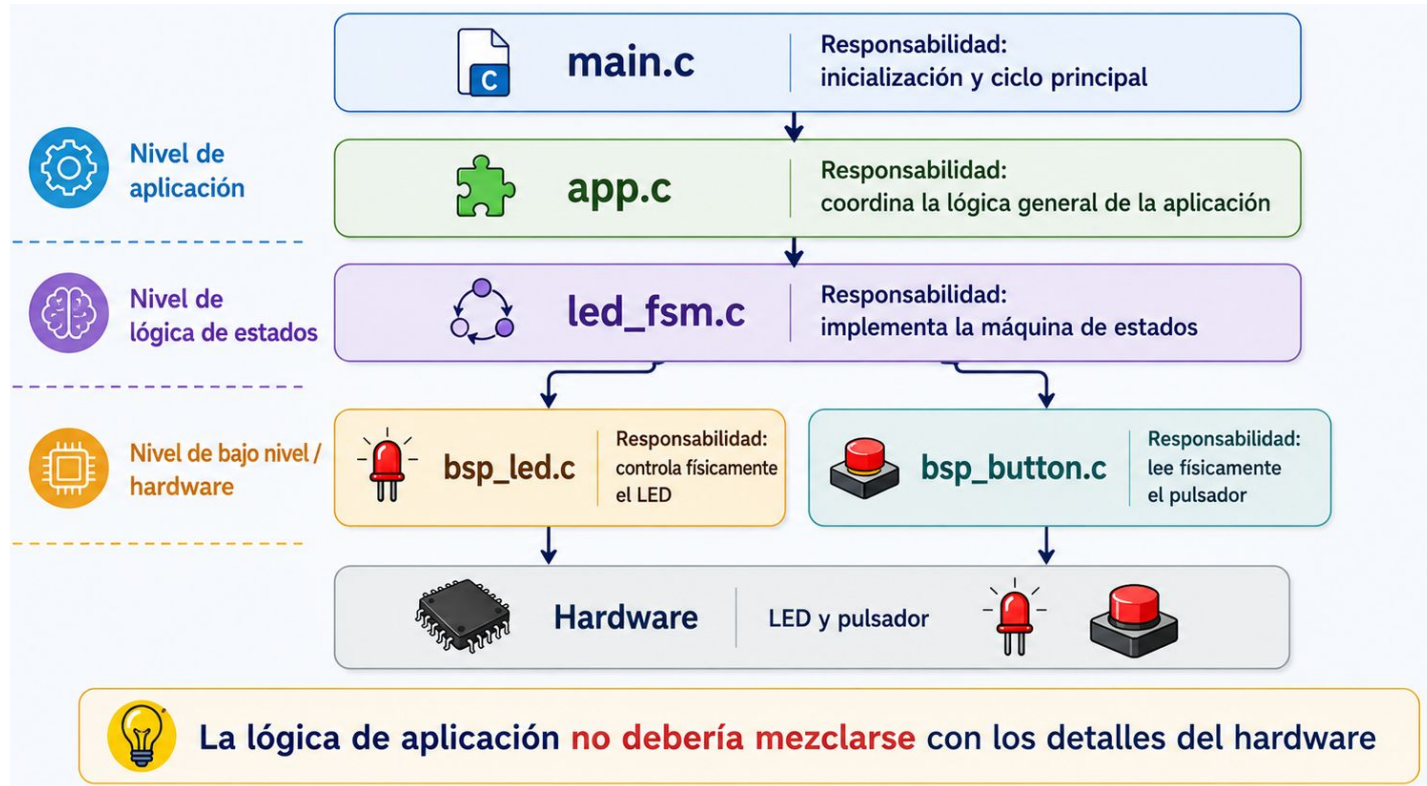
El problema: un `main.c` demasiado grande



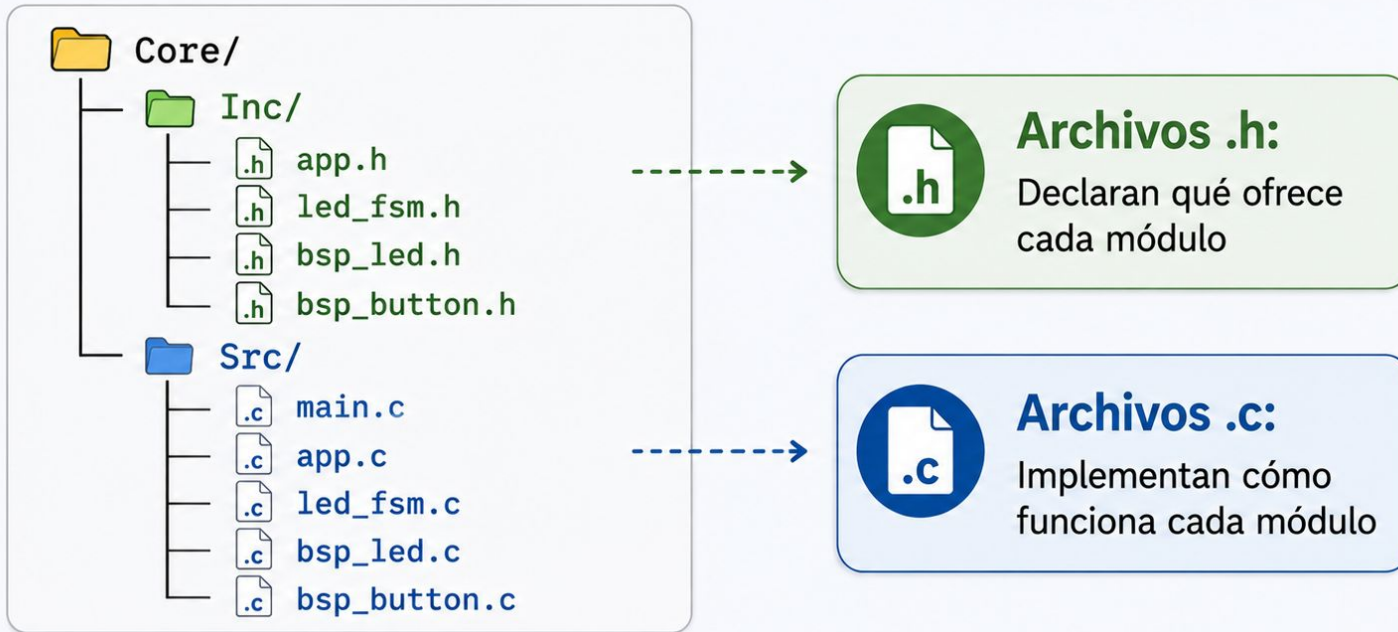
¿Qué es modularizar?



Separación de responsabilidades



Organización de archivos del proyecto



.h declara la **interfaz**; **.C** implementa el **comportamiento**

Interfaz pública e implementación interna



led_fsm.h

Interfaz pública

```
void led_fsm_init(void);  
  
void led_fsm_procesar_evento(evento_t evento);  
  
void led_fsm_actualizar(void);
```



Funciones que otros
módulos pueden usar



El resto del
programa usa
el **.h**, pero no
necesita conocer
todos los
detalles del **.c**



led_fsm.c

Implementación interna

```
static estado_led_t estado_actual;  
  
void led_fsm_init(void)  
{  
    estado_actual = ESTADO_APAGADO;  
    bsp_led_apagar();  
}
```

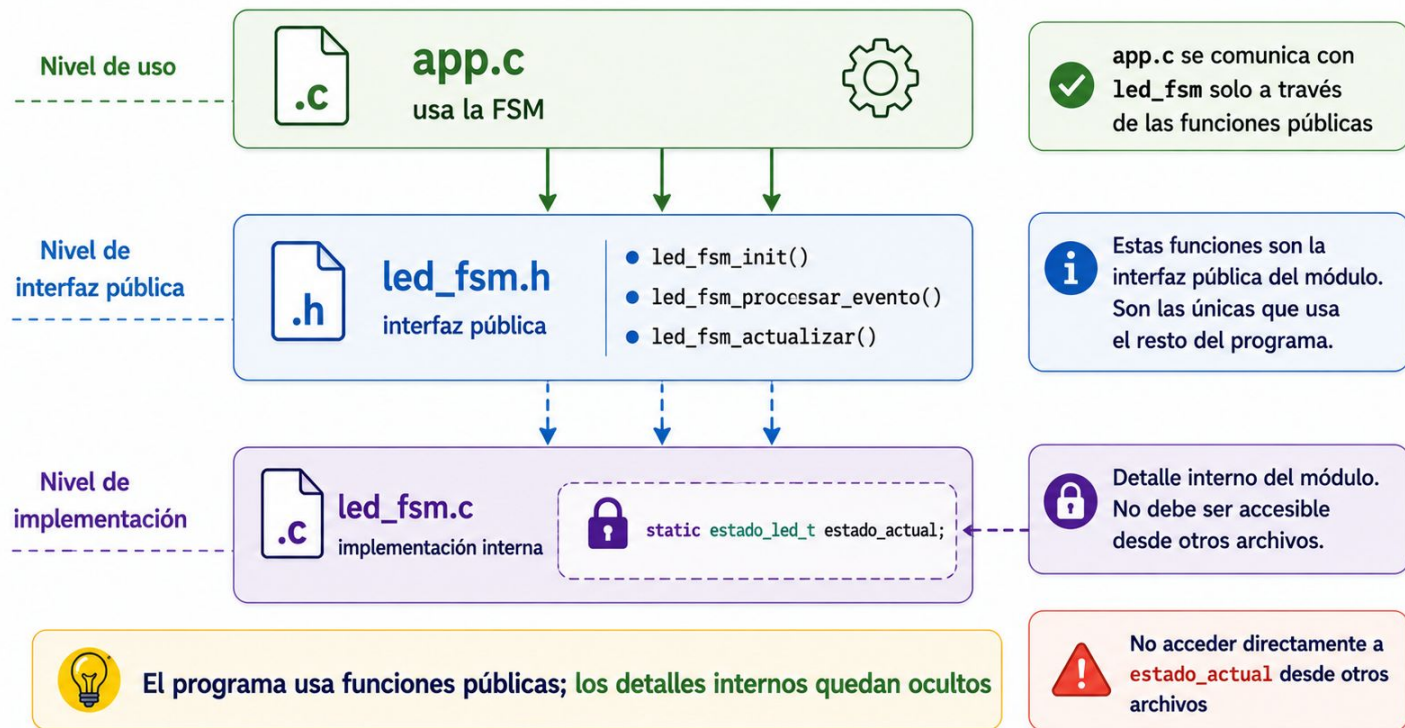


Detalles internos
del módulo



El **.h** declara **qué ofrece** el módulo; el **.c** define **cómo lo hace**

Encapsulamiento: usar un módulo sin conocer sus detalles internos



Encapsulamiento en módulos

